Punto 3

Verificamos que OpenSSH este instalado

```
vagrant@servidor1:~$ dpkg -l | grep openssh-server
ii  openssh-server                          1:8.9p1-3ubuntu0.14
  amd64        secure shell (SSH) server, for secure access from remote machines
vagrant@servidor1:~$
```

Verificamos el servicio SSH está activo

```
vagrant@servidor1:~$ sudo systemctl status ssh
● ssh.service - OpenBSD Secure Shell server
     Loaded: loaded (/lib/systemd/system/ssh.service; enabled; vendor preset: en>
     Active: active (running) since Tue 2026-04-07 16:12:36 UTC; 49min ago
       Docs: man:sshd(8)
             man:sshd_config(5)
   Main PID: 800 (sshd)
      Tasks: 1 (limit: 3425)
     Memory: 7.6M
        CPU: 2.128s
     CGroup: /system.slice/ssh.service
             └─800 "sshd: /usr/sbin/sshd -D [listener] 0 of 10-100 startups"
```

Verificamos que SFTP está habilitado

```
  GNU nano 6.2                          /etc/ssh/sshd_config
# override default of no subsystems
Subsystem       sftp    /usr/lib/openssh/sftp-server
```

Desde el cliente probamos que funcione

```
iente\parcial2> sftp vagrant@192.168.50.3
vagrant@192.168.50.3's password:
Connected to 192.168.50.3.
sftp>
```

Revisamos el estado de UFW y el puerto

```
vagrant@servidor1:~$ sudo ufw status
Status: active

To                         Action      From
--                         ------      ----
22                         ALLOW       Anywhere
22 (v6)                    ALLOW       Anywhere (v6)
```

Bloqueamos el puerto 22 para probar que falla

```
vagrant@servidor1:~$ sudo ufw deny 22
Rules updated
Rules updated (v6)
```

Probamos que efectivamente falla, ya que no ingresa

```
vagrant@servidor2:~$ sftp vagrant@192.168.50.3
```

Permitimos el acceso

```
vagrant@servidor1:~$ sudo ufw allow 22
Rules updated
Rules updated (v6)
```

Probamos que funciona

```
vagrant@servidor2:~$ sftp vagrant@192.168.50.3
vagrant@192.168.50.3's password:
Connected to 192.168.50.3.
sftp> exit
```

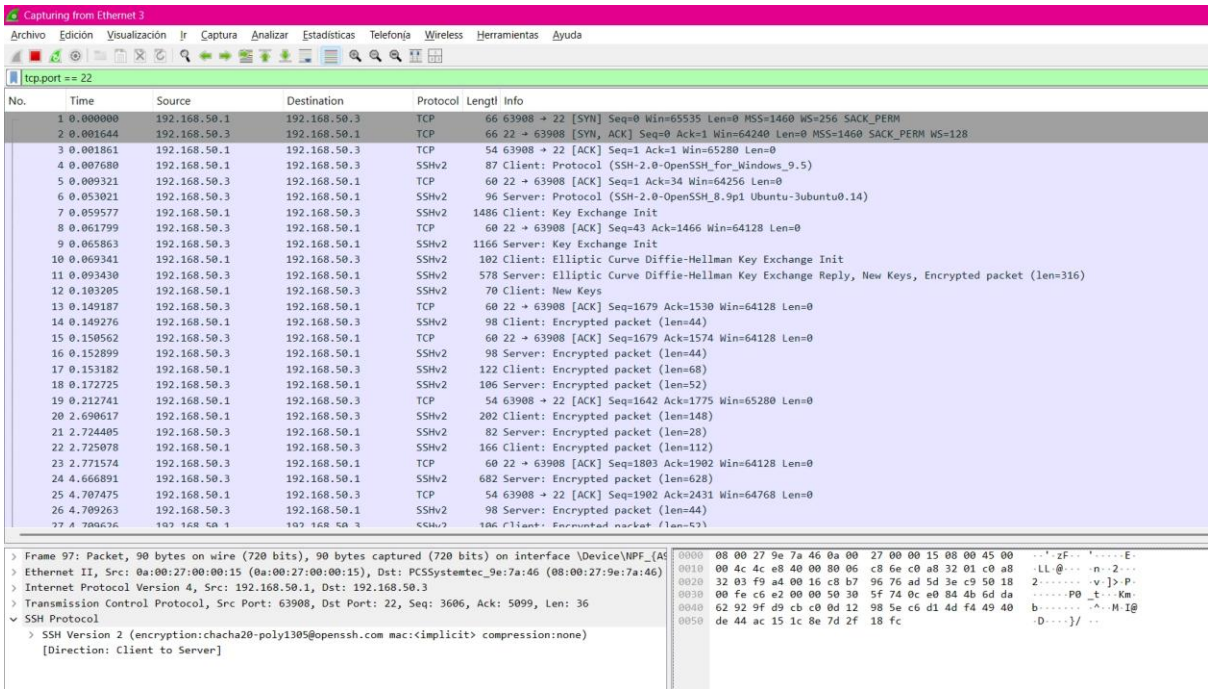
Una vez dentro, probamos los comandos solicitados

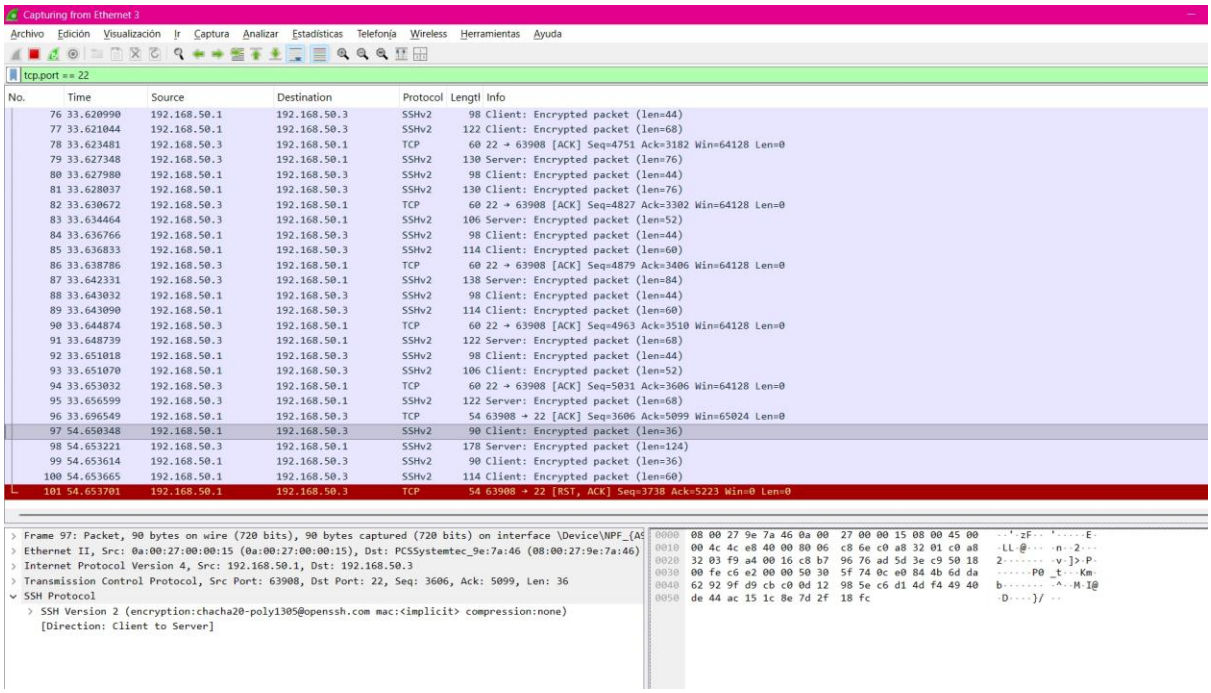
```
vagrant@servidor2:~$ echo "archivo parcial 2" > archivo.txt
vagrant@servidor2:~$ sftp vagrant@192.168.50.3
vagrant@192.168.50.3's password:
Connected to 192.168.50.3.
sftp> ls
sftp> put archivo.txt
Uploading archivo.txt to /home/vagrant/archivo.txt
archivo.txt
sftp> ls
archivo.txt
sftp> get archivo.txt
Fetching /home/vagrant/archivo.txt to archivo.txt
archivo.txt
sftp>
```

Y vemos que pasa si borramos el archivo

```
sftp> rm archivo.txt
Removing /home/vagrant/archivo.txt
sftp> get archivo.txt
File "/home/vagrant/archivo.txt" not found.
```

Capturamos con Wireshark el tráfico generado durante la sesión con el filtro tcp.port == 22





Viendo más de cerca tenemos:

SSHv2	70	Client: New Keys
TCP	60	22 → 63908 [ACK] Seq=1679 Ack=1530 Win=64128 Len=0
SSHv2	98	Client: Encrypted packet (len=44)
TCP	60	22 → 63908 [ACK] Seq=1679 Ack=1574 Win=64128 Len=0
SSHv2	98	Server: Encrypted packet (len=44)
SSHv2	122	Client: Encrypted packet (len=68)
SSHv2	106	Server: Encrypted packet (len=52)
TCP	54	63908 → 22 [ACK] Seq=1642 Ack=1775 Win=65280 Len=0
SSHv2	202	Client: Encrypted packet (len=148)
SSHv2	82	Server: Encrypted packet (len=28)
SSHv2	166	Client: Encrypted packet (len=112)

- En la columna de Protocolo se ve “SSHv2” eso muestra que está usando SFTP
- Se ve que está cifrado ya que sale:
  - New Keys: Se establecieron claves de cifrado
  - Encrypted packet: Todo lo que sigue va encriptado

Comparación con FTP:

En las capturas de tráfico FTP se observa que la comunicación inicia en el puerto 21 y posteriormente utiliza puertos adicionales (como el 50000) para la transferencia de datos. Además, se evidencian mensajes como “AUTH TLS” y “Client Hello”, lo que indica que el cifrado se negocia después de iniciar la conexión. En contraste, en las capturas de SFTP se identificó el uso exclusivo del puerto 22 bajo el protocolo SSHv2, donde se observa el intercambio de claves y posteriormente paquetes marcados como “Encrypted packet”. A diferencia de FTP, en SFTP no es posible visualizar información en texto plano como credenciales o comandos, ya que toda la comunicación (autenticación y datos) viaja cifrada dentro de un único canal seguro.

Tabla de comparación:

CRITERIO	FTPS	SFTP
Protocolo base	FTP + SSL/TLS	SSH
Número de puertos utilizados	21 + puertos dinámicos (ej. 50000)	Solo puerto 22
Tipo	Certificados SSL/TLS (OpenSSL)	Usa claves SSH
Modo de cifrado	Cifrado mediante SSL/TLS, después de negociación	Cifrado completo mediante SSH desde el inicio

Compatibilidad con firewalls	Baja (requiere múltiples puertos abiertos)	Alta (solo un puerto)
Facilidad de configuración	Más compleja (configuración de certificados y puertos)	Más sencilla (configuración directa con SSH)